Supplementary Material for the Article: Mechanically Verified Optimization and UPPAAL-Oriented Transformation of Timed Büchi Automata

ELIE FARES[1], JEAN-PAUL BODEVEIX[2], HUSSAIN AL-AQRABI[1] AND AZAR SALAMI[1]
[1]Higher Colleges of Technology, Sharjah, United Arab Emirates (e-mail: efares@hct.ac.ae, halaqrabi@hct.ac.ae, asalami@hct.ac.ae)
[2]IRIT, UPS, Université de Toulouse, France (e-mail: bodeveix@irit.fr)

ABSTRACT This supplement records extended implementation and reproduction details for the verified TBA optimization and UPPAAL export pipeline. The Rocq preservation theorems hold for every TBA represented by the formula-indexed clock interface, independently of whether the automaton was generated by the MTL translator; measurements are limited to benchmark automata recorded in the project.

S1. THE VERIFIED POST-PROCESSING PASSES

This section details the passes summarized in Table 1 of the article.

A. LOCATION INVARIANTS ON THE TRACE MODEL

A location invariant maps every clock x either to an upper bound $M \in \mathbb{R}$, standing for the constraint $x \leq M$, or to $+\infty$ (no constraint). We interpret automata with invariants on the extended words of Section II, as we do for automata without invariants. Along a run r, the automaton stays in location $r(i)$ between the events $i-1$ and i, and the invariant of $r(i)$ must hold during the whole stay, that is, at the clock values $\mathsf{val}_\rho(i,x) - \tau$ for every delay $\tau \in [0, \Delta_{i-1}]$, where $\Delta_{i-1} = t_i - t_{i-1}$ (and at the values $\mathsf{val}_\rho(0,x)$ for the initial location). This is the condition of Section II, read at the event that ends the stay. Since clocks increase while time elapses, an invariant made of upper bounds holds during the stay if and only if it holds at the event that ends it (lemma `inv_during_iff_at_event`). For the two-sided invariants of Section S1-C, a lower bound $x \geq m$ must likewise hold during the whole stay, hence at its beginning.

B. INVARIANT SYNTHESIS AND BACKWARD PROPAGATION

In the exported timed automaton, upper-bounded guards need invariants so that time does not elapse past all useful outgoing transitions. We compute location invariants from outgoing upper bounds and propagate successor requirements backward, following the standard timed-automata intuition of propagating admissible time domains [1].

a: Upper bound of a guard.

For a conjunctive guard g and a clock x, let $ub_x(g)$ be the smallest bound m of the constraints $x \leq m$, $x < m$, and $x = m$ occurring in g, and $+\infty$ if g contains none. Other constraints are ignored. A valuation satisfying g satisfies $x \leq ub_x(g)$.

b: Invariant synthesis.

For a location ℓ with outgoing transitions $\ell \xrightarrow{g_i, Z_i} \ell_i$, $i = 1, \ldots, m$, the synthesized invariant is

$$Inv(\ell)(x) = \max_{1 \leq i \leq m} ub_x(g_i),$$

with $Inv(\ell)(x) = +\infty$ when ℓ has no outgoing transition. Thus a clock that some outgoing guard does not bound does not appear in the invariant. The bound is always non-strict, even when it comes from a strict guard $x < m$: the invariant only needs to be implied by the guard of the next transition, and strictness is kept in the guards themselves.

Theorem S1.1 (Invariant synthesis)**.** *For every TBA $\mathcal{A}$, the automaton $\mathcal{A}$ with the synthesized invariants accepts the same timed words as $\mathcal{A}$.*

Proof. Adding invariants can only remove runs. Conversely, let r be an accepting run on ρ. At every event i, the transition taken from $r(i)$ is enabled, so its guard holds at the values $\mathsf{val}_\rho(i,x)$, which are therefore at most $Inv(r(i))(x)$. By the lemma `inv_during_iff_at_event`, the invariant of $r(i)$ holds during the whole stay. (Coq: `add_invariants_correct`.) □

c: Backward propagation.

One propagation step strengthens every transition $\ell \xrightarrow{g,Z} \ell'$ with the invariant of its target after the resets:

$$g := g \wedge Inv(\ell')[Z := 0],$$

where $Inv(\ell')[Z := 0]$ replaces every bound $x \leq M$ on a reset clock $x \in Z$ by the constant constraint $0 \leq M$, that is, by $\top$ if $M \geq 0$ and by $\bot$ otherwise. Invariants are then synthesized again from the strengthened guards.

Theorem S1.2 (Backward propagation)**.** *For every TBA $\mathcal{A}$ and every $n \in \mathbb{N}$, the automaton obtained by n propagation steps accepts the same timed words as $\mathcal{A}$.*

Proof. Strengthening guards can only remove runs. Conversely, consider an accepting run that takes $\ell \xrightarrow{g,Z} \ell'$ at event i and then a transition $\ell' \xrightarrow{g',Z'} \ell''$ at event $i+1$. The second guard holds at event $i+1$, so the values at that event are bounded by $Inv(\ell')$. Clock consistency gives $\mathsf{val}_\rho(i+1,x) = \Delta_i$ for $x \in Z$ and $\mathsf{val}_\rho(i+1,x) = \mathsf{val}_\rho(i,x) + \Delta_i \geq \mathsf{val}_\rho(i,x)$ otherwise; hence the strengthened guard $g \wedge Inv(\ell')[Z := 0]$ holds at event i. The argument uses clock consistency, so the theorem is stated on timed words. (Coq: `propagate_accepts` and `propagate_n_accepts`.) □

Each round of the optimization pipeline of Section III-G applies one propagation step. The tool repeats the rounds until the automaton is structurally equal to the previous one, as an OCaml value, or until 50 rounds have been performed; the bound guarantees termination, and the theorem holds for every number of steps, whether or not a fixpoint is reached. On our benchmarks, the rounds reach a fixpoint after at most six rounds (six for R6, at most five for every other completed configuration). The invariant computation at one location is summarized in the article's Figure 1.

d: Example.

We illustrate the algorithm on the automaton of Fig. S1, which contains a two-location cycle $\ell_1 \to \ell_2 \to \ell_1$ and a self-loop at ℓ_2. It has one clock x, the initial location ℓ_0, and the transitions

$$\ell_0 \xrightarrow{a,\ Z=\{x\}} \ell_1, \quad \ell_1 \xrightarrow{b,\ x\leq 2,\ Z=\emptyset} \ell_2,$$
$$\ell_2 \xrightarrow{c,\ \top,\ Z=\emptyset} \ell_1, \quad \ell_2 \xrightarrow{d,\ x\leq 1,\ Z=\emptyset} \ell_2.$$

The edge $\ell_1 \xrightarrow{b} \ell_2$ requires $x \leq 2$, and the self-loop at ℓ_2 requires the tighter bound $x \leq 1$.

1) *Synthesis.* $Inv(\ell_1)(x) = ub_x(x \leq 2) = 2$. At ℓ_2, $Inv(\ell_2)(x) = \max(1, +\infty) = +\infty$, because the transition toward ℓ_1 does not bound x. At ℓ_0, the only outgoing guard is $\top$, so $Inv(\ell_0)(x) = +\infty$.
2) *Propagation.* The transition $\ell_2 \xrightarrow{c,\ Z=\emptyset} \ell_1$ becomes guarded by $\top \wedge Inv(\ell_1) = (x \leq 2)$. The transition $\ell_0 \xrightarrow{a,\ Z=\{x\}} \ell_1$ receives $(0 \leq 2) \equiv \top$ and is unchanged.
3) *Synthesis.* At ℓ_2, the outgoing guards are now $x \leq 1$ and $x \leq 2$, so $Inv(\ell_2)(x) = \max(1, 2) = 2$.

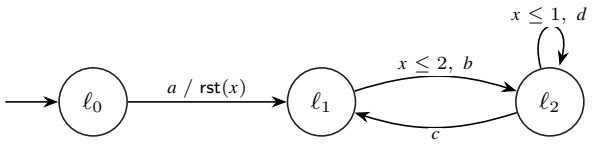


FIGURE S1. Automaton used to illustrate invariant propagation, with one clock *x* and actions *a–d*. Edges are labeled "guard, action / resets" (the guard is omitted when it is $\top$); only the edge $\ell_0 \to \ell_1$ resets *x*. The incoming arrow marks the initial location; acceptance plays no role here. The computed fixpoint is $Inv(\ell_1) = Inv(\ell_2) = (x \leq 2)$.

4) *Propagation.* The transition $\ell_1 \xrightarrow{b,\ x\leq 2} \ell_2$ receives $x \leq 2$, which it already contains, and the self-loop at ℓ_2 receives $x \leq 2$, which is implied by $x \leq 1$. Nothing changes.

The fixpoint is $Inv(\ell_0) = \top$ and $Inv(\ell_1) = Inv(\ell_2) = (x \leq 2)$. While the automaton remains in the cycle $\ell_1 \leftrightarrow \ell_2$, the clock x cannot exceed 2, and at every reachable valuation at least one outgoing transition of the current location is enabled. The invariants do not remove every blocking behavior: since x is never reset in the cycle, time is blocked once $x = 2$. They make the upper bounds of the property automaton explicit, and Theorems S1.1 and S1.2 guarantee that they do not change its language.

C. FORWARD PROPAGATION OF LOWER BOUNDS AND SIMPLIFICATION

a: Entry bounds.

Dually, for a transition $t = \ell \xrightarrow{g,Z} \ell'$ and a clock x, the *entry bound* of x along t is 0 if $x \in Z$ and otherwise the largest bound $lb_x(g)$ of the constraints $x \geq m$, $x > m$, and $x = m$ of g, if any. The entry bound of x at a location ℓ' is the minimum of the entry bounds of x along the transitions entering ℓ'; for the initial location, the initial entry, with bound 0, also counts. If some entering transition gives no bound on x, the clock x has no entry bound at ℓ'. Since clocks are nonnegative and do not decrease while time elapses, every clock with entry bound m at ℓ' satisfies $x \geq m$ during every stay in ℓ'.

b: Forward propagation.

The forward step adds $x \geq m$ to the guard of every transition leaving ℓ', for every clock x with entry bound m at ℓ' (Coq: `forward_accepts`). The entry bounds can also be read as lower-bound invariants: the two-sided automaton that carries the synthesized upper-bound invariants and the entry bounds as lower-bound invariants accepts the same timed words (`add_two_sided_invariants_accepts`). UPPAAL rejects lower-bound invariants, and their only role is to make infeasible transitions visible: after forward and backward propagation, a transition whose source location bounds a clock from below and whose target bounds it from above carries both bounds in its guard.

c: Simplification.

Three simplifications follow. First, a transition whose guard contains contradictory constraints is removed: a lower bound

and an upper bound on the same clock with $u < l$, or with $u = l$ and one of them strict, or an upper bound $x \leq u$ with $u < 0$ or $x < u$ with $u \leq 0$. Such a transition is never enabled (Coq: `remove_contradictory_accepts`). Second, the transitions leaving a non-initial location that has no entering transition are removed: such a location never occurs on a run after position 0 (Coq: `remove_unreachable_accepts`). The iteration of the pipeline removes chains of such locations one step at a time. A location whose only entering transitions are self-loops keeps them, but it is unreachable from the initial location; the tool counts and prints only the locations reachable from the initial one and their transitions. Third, a transition whose action label is unsatisfiable, because it requires two different actions or an action and its negation, is removed (Coq: `remove_unsat_accepts`). Such labels arise because the translator treats an action a and the atom $\neg a$ of $\mathcal{T}(f)$ as independent propositions; removing them also enables clock merging, since the resets of a transition that can never be taken would otherwise prevent it.

D. DEAD RESETS AND MERGING OF SYNCHRONOUSLY RESET CLOCKS

a: Dead resets.

A clock x is *live* at a location ℓ if some transition that tests x in its guard can be taken from ℓ, possibly after other transitions. The live locations are computed as a backward closure, along all transitions, from the sources of the transitions that test x; the other locations are *dead* for x. Since the correctness of the pass does not depend on how this closure is computed, the pass checks the two properties that the proof uses, namely that no transition leads from a dead location to a live one and that none tests x from a dead location, and then removes x from the reset sets of the transitions that enter a dead location. After such a transition, the value of x is never read again, so whether it is reset does not matter. The proof builds, from every accepted extended word, an extended word that differs only in the reset decisions of x, and conversely (Coq: `remove_dead_resets_clock_accepts`). The pass is applied to every clock (`remove_dead_resets_accepts`).

b: Synchronous clocks.

The verified translation shares the clock of syntactically identical timed subformulas. Clocks of *different* subformulas can also be merged when they are reset synchronously. Two clocks $x \neq y$ are *mergeable* if

(i) every transition resets x if and only if it resets y;
(ii) no transition enters the initial location; and
(iii) every transition leaving the initial location resets x and does not test it.

Then, after the first event, x and y always have the same value, and every test of x can be replaced by the same test on y (Coq: `merge_accepts`). The condition is checked by a Boolean function proved to imply it, and the pass tries all pairs of reset clocks (`merge_all_accepts`). The merged clock x is no longer tested, so its remaining resets are removed by the dead-reset pass in the next round. This verified merging is restricted to synchronously reset clocks; merging clocks whose values coincide for other reasons is not covered.

E. NORMALIZATION OF GUARDS

Backward and forward propagation add constraints to guards. Every time a guard is strengthened, it is *tightened*: a constraint implied by another constraint of the same guard on the same clock is dropped, so that a guard keeps at most one lower bound and one upper bound per clock (an equality counting as both); for instance, $x \leq 5 \wedge x < 2$ becomes $x < 2$. The implication test compares bounds and strictness, and is proved sound (Coq: `tighten_holds`). Normalization tightens every guard and also removes the lower bounds $x \geq m$ with $m \leq 0$ and $x > m$ with $m < 0$, which every clock value satisfies; it is needed after the merging of clocks, which renames constraints. The removal of trivial lower bounds relies on the nonnegativity of clock values, so the theorem is stated on timed words (Coq: `normalize_accepts`).

F. MERGING OF TRANSITIONS AND OF LOCATIONS

a: Transitions.

Two transitions with the same source, target, and resets are merged when the union of their enabling conditions is again a conjunction. A transition is removed when another transition with the same source, target, and resets is *weaker*, that is, when its label and guard imply those of the other transition, constraint by constraint (subsumption). Two transitions whose labels, or guards, differ only by a literal and its negation, such as p and $\neg p$, $x < 2$ and $x \geq 2$, or $x \leq 5$ and $x > 5$, are replaced by one transition without that literal (resolution). The pass preserves the language because every enabled transition has an enabled counterpart with the same source and target, and conversely (Coq: `merge_trans_accepts`). The other unions are disjunctive; they are formed by the export (Section S1-G).

b: Locations.

A location ℓ' is merged into a location ℓ when both are accepting or both are not, and when every transition leaving ℓ' has a twin leaving ℓ, and conversely: same label, guard, and resets, compared as sets, and same target once ℓ' is replaced by ℓ. The transitions entering ℓ' are redirected to ℓ, and those leaving ℓ' are removed. A run of the original automaton becomes a run of the merged one by replacing ℓ' by ℓ; conversely, a run of the merged automaton is followed in the original one, taking in ℓ' the twin of the transition taken in ℓ (Coq: `merge_states_all_accepts`; "state" in a Coq name means location). The condition on the Büchi marking is necessary: merging an accepting location with a non-accepting twin would accept runs that visit only the latter infinitely often.

G. TRANSITION MERGING AND ELIMINATION OF DISJUNCTIVE INVARIANTS

The export works on a model whose guards are in disjunctive normal form over clock constraints and clock-difference

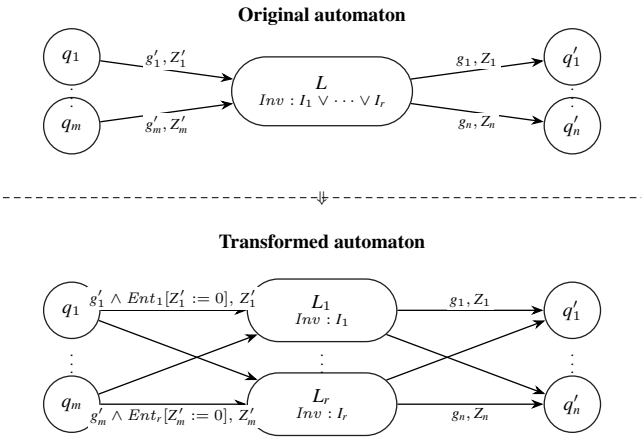


FIGURE S2. General splitting of a location with a disjunctive invariant $I_1 \vee \cdots \vee I_r$ into one copy per disjunct.

constraints $x - y \leq b$, and whose location invariants are disjunctions of conjunctions of upper bounds. It is interpreted on the same trace model as before. The export merges transitions, splits locations, splits disjunctive guards, normalizes the result, and merges the initial location with an equivalent one, each pass being proved to preserve the language of timed words; it ends with an automaton whose guards and invariants are conjunctive, as UPPAAL requires (Coq: `export_accepts`, `export_guards_conjunctive`, and `export_invariants_conjunctive`).

a: Transition merging.

Transitions with the same source, target, action label, and reset set are merged into one transition whose guard is the disjunction of their guards (Coq: `merge_transitions_accepts`). The invariant of a location ℓ is then naturally disjunctive: it has one disjunct $ub(g)$, the conjunction of the upper bounds of g, for every conjunct g of every guard leaving ℓ:

$$I = I_1 \vee \cdots \vee I_r.$$

A disjunct that implies another disjunct is redundant and is dropped. The resulting invariant is more precise than the clock-by-clock maximum of Section S1-B, which is its conjunctive over-approximation.

b: Splitting locations.

UPPAAL does not allow disjunctive location invariants: invariants must be conjunctions of clock constraints defining convex, past-closed zones. A location with invariant $I_1 \vee \cdots \vee I_r$ is therefore replaced by locations with conjunctive invariants and the same timed behavior. Bodeveix and Filali studied disjunctive invariants in UPPAAL [2]. Figure S2 summarizes the transformation.

The location L is replaced by one copy L_k for each disjunct I_k, with the conjunctive invariant I_k. Every transition entering L is replicated toward every copy L_k, and its guard is strengthened with the entry condition Ent_k after the resets; every transition leaving L is replicated from every copy. The disjunctive invariant itself is never built: the copies introduce the conjunctive invariants directly, and the theorem compares the split automaton with the automaton without invariants. The run starts in a dedicated copy of the initial location, without an invariant, since it is not entered by a transition. In the Coq development, the copy k of location ℓ is numbered $\ell \cdot \kappa + k$, where $\kappa - 1$ is the largest number of disjuncts and the copy $\kappa - 1$ is the initial one; numbers that correspond to no disjunct are unreachable locations.

c: Entry condition.

The disjuncts need not be mutually exclusive, so simple replication would introduce nondeterministic choices and could select a copy that permits less time to elapse than another one. The entry condition therefore selects a copy with the *longest* admissible delay. The synthesized invariants have non-strict bounds only (Section S1-B); let

$$I_k = \bigwedge_{c \in C_k} c \leq m^k_c.$$

From a valuation v, I_k admits the delays up to $Del_k(v) = \min_{c \in C_k}(m^k_c - v(c))$, with $Del_k(v) = +\infty$ when C_k is empty. The entry condition of L_k requires I_k and $Del_l(v) \leq Del_k(v)$ for every other copy l. The comparison $Del_l \leq Del_k$ holds when some bound $c' \in C_l$ is reached no later than every bound $c \in C_k$, which is expressed with clock differences:

$$Ent_k \triangleq I_k \wedge \bigwedge_{l \neq k} \bigvee_{c' \in C_l} \bigwedge_{c \in C_k} (c - c' \leq m^k_c - m^l_{c'}).$$

When C_k is empty, the copy admits every delay and, since redundant disjuncts are removed beforehand, no other C_l is empty, so that the conjunction over l is true. Ties are allowed: when two copies admit the same delay, both entry conditions hold.

Theorem S1.3 (Elimination of disjunctive invariants)**.** *The split automaton accepts the same timed words as the automaton without invariants.*

Proof. Every copy L_k behaves as L with additional constraints, so the split automaton accepts no new word. Conversely, consider an accepting run that enters L with the valuation v (after the resets) and leaves it after a delay τ along a transition whose guard has a conjunct g. This conjunct holds at $v+\tau$, hence so does the disjunct $ub(g) = I_j$, and $Del_j(v) \geq \tau$. If I_j was dropped because it implies a kept disjunct, that disjunct admits at least the same delays, so we may assume that I_j is kept (Coq: `disj_inv_cover`). Choose a copy k whose delay $Del_k(v)$ is maximal (Coq: `best`). Its entry condition holds at v, and $Del_k(v) \geq Del_j(v) \geq \tau$, so I_k holds during the whole stay. The successor copies are chosen in the same way at every position. Clock-difference constraints are invariant under the elapse of time, so Ent_k can be checked on the valuation after the resets, and $Ent_k[Z := 0]$ on the valuation before them. (Coq: `split_accepts`.) □

The substitution $[Z := 0]$ maps a difference $c - c' \leq b$ to $0 \leq b$ when both clocks are reset, to $c' \geq -b$ when only c is reset, and to $c \leq b$ when only c' is reset. The disjunctions over c' in Ent_k, and those created by transition merging, are finally eliminated by putting the guards into disjunctive normal form and splitting every transition into one copy per disjunct (Coq: `explode_accepts`). In the worst case, the number of copies is the product of the sizes of the disjunctions.

d: Final normalization.

The guards are normalized before and after the split (Coq: `normalize_d_accepts`): in every conjunction, constraints implied by another one are removed, as in Section S1-E, together with the trivial differences $x - x \leq b$ with $b \geq 0$; conjunctions containing a difference $x - x \leq b$ with $b < 0$ are unsatisfiable and are removed; and a conjunction that implies another conjunction of the same guard is removed. Finally, the transitions leaving a location that is neither initial nor the target of a transition are removed, until nothing changes (Coq: `dprune_n_accepts`); this removes in particular the transitions of the copies that correspond to no disjunct. The dedicated copy of the initial location is then merged into another location when both have the same acceptance status, equivalent invariants (an invariant with an empty disjunct being equivalent to no invariant), and the same outgoing transitions, compared as sets after redirection; this is the case, for instance, when the initial location has no invariant (Coq: `dmerge_states_all_accepts`). The resulting automaton has conjunctive guards with difference constraints and conjunctive location invariants made of upper bounds.

e: Symbolic transitions.

Each transition of the exported automaton reads a cube: a conjunction of action literals and a conjunction of clock constraints. For the drawing and for the comparison with tools whose transitions carry Boolean formulas, the last pass merges all the transitions with the same source, target, and resets into one *symbolic transition*, labeled by a disjunction of cases $E_1 \wedge c_1 \vee \cdots \vee E_k \wedge c_k$, where E_j is a set of events, either finite or the complement of a finite set, and c_j a conjunction of clock constraints; the cases with the same constraints are merged by union of their sets of events. A symbolic transition is enabled when the current event belongs to some E_j whose constraints c_j hold, and its resets match those of the extended word. The symbolic automaton accepts the same extended words as the exported one, hence the same timed words for both the existential and the conventional acceptance (Coq: `symbolic_ext_accepts`, `MTL_to_symbolic_correct_with`, and `MTL_to_symbolic_correct0_with`). The UPPAAL model keeps the conjunctive transitions, since a UPPAAL edge synchronizes on one channel and its clock guard must be a conjunction.

S2. IMPLEMENTATION AND REPRODUCTION DETAILS

The Rocq definitions and preservation proofs are in `proof/`; the optimizer and exporter functions are extracted into `tool/src/optim.ml`. The parser, Spot invocation, LBTT reader, fixed-point driver, and XML/DOT printers are hand-written. The importer, extraction cleanup script, OCaml runtime, external tools, and XML serialization remain outside the kernel-checked transformation theorems.

The upstream-generated measurements retain five runs per attempted configuration, each limited to 300 seconds. The current summary has 31 attempted configurations and 29 completed optimizer inputs. Machine details are in `evaluation/machine.txt`; raw runs and summary rows are in `evaluation/ours\_runs.tsv` and `evaluation/ours\_results.tsv`. The current R2 input, optimized graph, and export are in `evaluation/generated\_examples/R2/`, with command and provenance recorded in its README.

The Rocq transformations use existential initial clock values. UPPAAL starts clocks at zero, so deployment through the formula-indexed workflow relies on the proved `init_free` condition. The lemma `init_free_accepts0` applies to every exported automaton on which the check succeeds; the check may fail on an external automaton whose acceptance relies on nonzero initial clock values.

S3. PASS-LEVEL AND ABLATION DATA

The sixteen directly constructed records and larger parameterized families are defined in `evaluation/synthetic\_tba\_bench.ml`. These values inhabit the extracted `Optim.tBA` type; no third-party parser or external corpus is involved. `evaluation/synthetic\_tba\_passes.tsv` records the input, each operation in every changed round, the optimized or capped result, and explicit export. It records declared and graph-reachable locations separately, along with transitions, active clocks, reset occurrences, guard atoms, invariant counts, diagonal guards, elapsed time, and OCaml heap-capacity observations. `evaluation/synthetic\_tba\_memory.tsv` records peak resident set size from a separate process per small model. The corrected full trace and mechanism examples are preserved under `evaluation/runs/2026-10-10-synthetic-ablation-v6/`.

The leave-one-operation-out driver in `evaluation/synthetic\_tba\_ablation.tsv` repeats the remaining composite operations to stability for each variant. The complete variant is structurally compared with the extracted optimizer. This comparison validates the measurement harness's structural correspondence; it is neither an independent semantics proof nor a theorem for the altered pipelines. `evaluation/synthetic\_tba\_parameterized.tsv` gives intermediate sizes for duplicate-edge, unreachable-cascade, redundant-bound, synchronous-reset, and disjunctive-invariant families. The 50-round cap probe is preserved under `evaluation/runs/2026-10-10-round-cap-v4/`; size 51 changes

TABLE S1. Transitions before and after optimization. Comp.: explicit transitions of the optimizer input; Sym. no opt. and Sym. opt.: symbolic transitions of the exported automaton with the optimizer disabled (`-n 0`) and enabled; Opt.: explicit transitions after optimization; CASAAL: symbolic transitions of its output with mutually exclusive events (– if not encodable). A bound of 300 s means that the unoptimized export did not finish within 300 seconds. R7 and R8 do not complete.

ID	Comp.	Sym. no opt.	Sym. opt.	Opt.	CASAAL
F1	5	4	4	5	4
F2	5	5	1	1	1
F3	5	5	5	5	–
F3b	1	1	1	1	1
F4	5	4	4	5	4
F5	38	34	25	25	15
F6	38	34	25	25	15
F7	4	4	4	4	4
F8	4	4	4	4	4
F9	9	9	9	9	9
F10	13	12	12	13	12
F11	11	11	7	7	7
F12	17	17	9	9	9
F13	17	17	8	8	–
F14	26	20	14	17	18
R1	5	4	4	5	4
R2	25	16	12	16	12
R3	125	64	32	44	32
R4	625	256	80	112	80
R5	3125	>300 s	192	272	192
R6	15625	>300 s	448	640	448
N1	5	5	5	5	5
N2	11	9	7	9	7
N3	21	14	9	16	9
N4	36	20	11	25	11
N5	57	27	13	36	13
N6	85	35	15	49	15
N7	121	44	17	64	17
N8	166	54	19	81	19

after round 50, and size 64 continues changing after rounds 51 and 52. All these inputs are synthetic.

S4. SYMBOLIC TRANSITION COUNTS BEFORE AND AFTER OPTIMIZATION

Table S1 gives, for every benchmark configuration that completes, the explicit transitions of the optimizer input and of the optimized automaton, and the symbolic transitions of the exported automaton with the optimizer disabled and enabled. Both symbolic counts apply the same grouping, so their difference is attributable to the optimizer. The values are produced by `evaluation/make_symbolic_before_after.py` from the raw output of `evaluation/run_symbolic_before_after.sh`. With the front end of the translation article (`-noweak -norecur`) and the optimizer disabled, the symbolic count equals the number of edges obtained by grouping the transitions of `compile`(f) by source, target, guard, and resets, as counted in that article, on all 23 configurations that complete within 300 s (F1–F14, R1–R4, N1–N8); R5 does not finish in this setting and R6–R8 do not build. The check is `evaluation/run_chain.sh`, with output in `evaluation/chain_noweak.tsv`.

S5. REPRODUCING SYNTHETIC MEASUREMENTS

From the repository root under WSL, the extracted-code measurements can be regenerated with:

```
ocamlc -w -a -c tool/src/optim.ml \
  -o tmp/optim.cmo
ocamlc -w -a -I tmp \
  -o tmp/synthetic_tba_bench.exe \
  unix.cma tmp/optim.cmo \
  evaluation/synthetic_tba_bench.ml
tmp/synthetic_tba_bench.exe \
  > evaluation/synthetic_tba_passes.tsv
tmp/synthetic_tba_bench.exe --ablation \
  > evaluation/synthetic_tba_ablation.tsv
tmp/synthetic_tba_bench.exe --parameterized \
  > evaluation/synthetic_tba_parameterized.tsv
bash evaluation/run_synthetic_tba_memory.sh
python evaluation/make_synthetic_tba_table.py
python evaluation/make_synthetic_tba_ablation_table.py
python evaluation/make_parameterized_summary.py
python evaluation/make_optimizer_aggregate_table.py
python evaluation/make_parameterized_trace_table.py
```

The profiled full pass sequence is structurally compared with extracted `optimize_step`; these runs measure the extracted implementation and do not compile Rocq proof source. The leave-one-operation-out variants are measurement harnesses, not separately proved optimizers.

S6. ARCHIVED UPSTREAM DIAGNOSTICS

Repository files retain formula-to-automaton timing, Spot representation, CASAAL, and cross-tool diagnostics under `evaluation/`. Those measurements concern upstream translation and its validation rather than the optimizer. They are excluded from this supplement to avoid attributing upstream preprocessing or backend behavior to the optimization contribution; they remain available as artifact data.

S7. UPPAAL CASE-STUDY ARTIFACTS

The two system families, four configurations, UPPAAL queries, and verdicts are under `evaluation/case_study/`. The eight queries were rerun with UPPAAL 5.0.0 `verifyta` (`uppaal_results_rerun_2026-10-11.txt`) and agree with the archived verdicts (`uppaal_results.txt`); the results contain no before/after runtime, memory, or zone counts. The matched TChecker artifacts are under `evaluation/runs/2026-10-10-tchecker-before-after-v3/`. They report equal before/after verdicts and exploration on two response-system cases whose exported XML is byte-identical. TChecker timed-Büchi emptiness is not ordinary UPPAAL reachability, and the XML-to-TChecker adapter is not verified.

REFERENCES

[1] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on UPPAAL," in *Formal Methods for the Design of Real-Time Systems*, ser. Lecture Notes in Computer Science. Springer, 2004, vol. 3185, pp. 200–236.

[2] J.-P. Bodeveix and M. Filali, "About disjunctive invariants in UPPAAL," in *15th Nordic Workshop on Programming Theory (NWPT)*, Turku, Finland, Oct. 2003, workshop presentation, 29–31 October 2003. [Online]. Available: http://www.abo.fi/~Kaisa.Sere/15NWPT.html